

APEC 8222 Assignment 2

Python Basics and GitHub Pages Website

Phillipa Gowdy-Jaehnig

2025-11-11

Assignment 2: Python Basics and GitHub Pages Website

For this assignment, see the QMD or IPYNB files in our course repository named 02-python_basics.qmd or 02-python_basics.ipynb.

Most things in this can be solved using methods shown in lecture. However, some cases will require you to search the internet for answers.

Question 1

a). **Indentation and Pythonic Code:** - Create a function named `print_numbers` that prints numbers from 1 to 10, each on a new line. Ensure your code is properly indented and follows Pythonic conventions.

```
#1 a)
def print_numbers():
    """ Prints numbers from 1 to 10, each on a new line. """
    for i in range(1, 11):
        print(i)
```

b.) **Rules of Functions and Comments:** - Define a function named `calculate_area` that takes two arguments `length` and `width`, and returns the area of a rectangle. Include a docstring explaining what the function does. See the lecture notes for an example of a docstring.

```
#1 b)
def calculate_area(length, width):
    """Calculates the area of a rectangle. Takes two arguments `length` and `width`, and returns the area.
    return(length * width)
```

c.) **Basic Data Types:** - Create a variable `is_active` and set it to `True`. Print the type of `is_active` using the `type()` function.

```
#1 c) Answer:
is_active = True
type(is_active)
```

`bool`

d.) **Working with Lists:** - Create a list named `fruits` containing some fruit names. Use the `append` method to add 'Mango' to the list and print the updated list.

```
#1 d) Answer:
fruits = ["Banana", "Passionfruit", "Kiwi", "Lychee"]
fruits.append("Mango")
print(fruits)
```

`['Banana', 'Passionfruit', 'Kiwi', 'Lychee', 'Mango']`

Question 2:

a.) **Naming Conventions:** - Create a variable following the `snake_case` naming convention to store the number of days in a week.

```
#2 a) Answer:
import calendar
days_in_a_week = len(calendar.day_name)
print(days_in_a_week)
```

`7`

b.) **Control Flows - Conditionals:** - Write a simple program that checks if a number is positive, negative, or zero using `if`, `elif`, and `else` statements.

```
#2 b) Answer:
import math
i = math.pi

if i == 0:
```

```

    print(i, " equals zero.")
elif i < 0:
    print(i, " is negative.")
else:
    print(i, " is positive.")

```

3.141592653589793 is positive.

c.) **Control Flows - Loops:** - Write a for loop to calculate the sum of numbers from 1 to 100.

```

#2 c) Answer:
n = 0
for i in range(0, 101):
    n += i
print(n)

```

5050

d.) **Control Flows - Loop Control Statements:** - Modify the loop from the previous sub-question to stop the loop if the sum becomes greater than 1000 using a **break** statement.

```

#2 c) Answer:
n = 0
for i in range(0, 101):
    n += i
    if n > 1000:
        break
print(n)

```

1035

e.) **Functions - Arguments and Return Values:** - Create a function named `get_full_name` that takes two arguments `first_name` and `last_name`, and returns the full name.

```

#2 e) Answer:
def get_full_name(first_name, last_name):
    """Takes two arguments `first_name` and `last_name`, and returns the full name on one line
    print(f"{first_name} {last_name}")

```

Question 3

Part A: Write a python function that calculates the sum of all squared numbers from 1 to x. Illustrate that it works for $x = 20$.

HINT, `**` is the exponent operator in python. HINT syntax for a python function is:

`def function_name(variable_name): outcome = variable_name + 1 return outcome`

#3A Answer

```
def sum_squares_until(x: int) -> int: #Ensures input and output are integers
    """Calculates the sum of all squared integers from 1 to x. Takes an input positive integer
    sum_squares = 0
    for i in range(1, x+1):
        sum_squares += i**2
    return sum_squares
```

```
sum_squares_until(20)
```

#Here's a function that works with negative integers as well,

```
def sum_squares_between(x: int) -> int: #Ensures input and output are integers
    """Calculates the sum of all squared integers from 1 to x. Takes an input integer x and o
    sum_squares = 0
    if x < 1:
        for i in range (x, 2):
            sum_squares += i**2
        return sum_squares
    else:
        for i in range(1, x+1):
            sum_squares += i**2
        return sum_squares
```

```
sum_squares_between(20)
```

```
sum_squares_between(-5)
```

56

The python library named “os” is a built-in library for dealing with any file on your operating system. Often, research tasks involve LOTS of files and you need to iterate over them. To show you know how to do this, use the `os.listdir` function to answer the following questions.

Note that you need to write “import os” to import the library into your code before you can use it:

Part B: Print out a list of all the files your class root directory. I don’t care how many are actually there (in case you’ve added some yourself) but show me how.

#3B Answer

```
import os
```

```
#First, get path to current working directory
current_directory = os.getcwd()
```

```
#Because this file is in my "assignment" folder rather than my class root directory, I need to
class_root_directory = os.path.dirname(current_directory)
```

```
print(f"Listing contents of: {class_root_directory}\n")
```

```
#Next, get a list of all entries.
```

```
# Use the try ... except method to handle errors so the program doesn't crash
try:
```

```
    contents = os.listdir(class_root_directory)
    print(contents)
```

```
except OSError as e:
```

```
    print(f"Error accessing directory: {e}")
```

Listing contents of: C:\Users\phill\Downloads\apex_8222\apex_8222_2025_pgj

['.git', '.gitignore', 'assignments', 'environment.yml', 'first_script.py', 'gdal_rasters_pgj']

Part C: Using a FOR loop, iterate over the list from above. Using the function len(), count how many letters there are in the filenames. HINT just like in real life, you may need to google the len() function and see how it work on e.g. strings.

#3C Answer

```
#First answer assumes all entries in the directory are files
```

```
for filename in contents:
```

```
    len(filename)
```

```
#First, get path to current working directory
```

```
current_directory = os.getcwd()
```

```

#Because this file is in my "assignment" folder rather than my class root directory, I need to
class_root_directory = os.path.dirname(current_directory)
print(f"Filename length in {class_root_directory}\n")

#Next, get a list of all entries.
# Use the try ... except method to handle errors so the program doesn't crash
try:
    contents = os.listdir(class_root_directory)
# count how many letters there are in the filenames
    for filename in contents:
        print(len(filename))
except OSError as e:
    print(f"Error accessing directory: {e}")

```

Filename length in C:\Users\phill\Downloads\apex_8222\apex_8222_2025_pgj

4
 10
 11
 15
 15
 22
 10
 16
 16
 13
 9
 16

Part D: Write a Python program which iterates over the integers from 1 to 50. For multiples of three print “Fizz” instead of the number and for the multiples of five print “Buzz”. For numbers which are multiples of both three and five print “FizzBuzz”. Hint: look up how to use the modulo operator (which is written as % in python) which would let you test when the remainder of a division is exactly 0.

Side-note, this is a hilariously over-used question that most software engineers get asked on their first interview for a job.

```

# 3D Answer
for i in range(1,51):
    # Print "Fizzbuzz" if the integer is evenly divisible by both 3 and 5.

```

```
if i % 3 == 0 and i % 5 == 0:
    print("Fizzbuzz")
# Print "Fizz" if the integer is evenly divisible by 3 only.
elif i % 3 == 0:
    print("Fizz")
# Print "Buzz" if the integer is evenly divisible by 5 only.
elif i % 5 == 0:
    print("Buzz")
# Print the integer if it is evenly divisible by neither 3 nor 5.
else:
    print(i)
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
Fizzbuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
Fizzbuzz
```

31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
Fizzbuzz
46
47
Fizz
49
Buzz

Question 4

Create a personal website using GitHub Pages. The website should be just a few pages, including a landing page, a research page, and a contact page. Use Markdown to format the content on your pages. Include links to your GitHub profile and any relevant projects or repositories you have worked on. Once completed, share the link to your GitHub Pages website here. Note, for the pages to actually be hosted online, you will need to create a new repository named `.github.io` and push your quarto-rendered markdown files there. See GitHub Pages documentation for more details.

<https://phillipagj.github.io/>